

CHAPTER 10

ESTABLISHING REQUIREMENTS

[10.1 Introduction](#)

[10.2 What, How, and Why?](#)

[10.3 What Are Requirements?](#)

[10.4 Data Gathering for Requirements](#)

[10.5 Data Analysis, Interpretation, and Presentation](#)

[10.6 Task Description](#)

[10.7 Task Analysis](#)

Objectives

The main aims of this chapter are to:

- Describe different kinds of requirements.
- Enable you to identify different kinds of requirements from a simple description.
- Explain how different data gathering techniques (those introduced in [Chapter 7](#) and others) may be used during the requirements activity.
- Enable you to develop a scenario, a use case, and an essential use case from a simple description.
- Enable you to perform hierarchical task analysis on a simple description.



10.1 Introduction

An interaction design project may aim to replace or update an established system, or it may aim to develop a totally innovative product with no obvious precedent. There may be an initial set of requirements, or the project may have to begin by producing a set of requirements from scratch. Whatever the initial situation and whatever the aim of the project, the users' needs, requirements, aspirations, and expectations have to be discussed, refined, clarified, and probably re-scoped. This requires an understanding of, among other things, the users and their capabilities, their current tasks and

goals, the conditions under which the product will be used, and constraints on the product's performance.

Establishing requirements is not simply writing a wish list of features. Given the iterative nature of interaction design, isolating requirements activities from design activities and from evaluation activities is a little artificial, since in practice they are all intertwined: some design will take place while requirements are being established, and the design will evolve through a series of evaluation–redesign cycles. However, each of these activities can be distinguished by its own emphasis and its own techniques.

This chapter provides a more detailed overview of establishing requirements. We introduce different kinds of requirements and explain some useful techniques.

10.2 What, How, and Why?

10.2.1 What Are We Trying to Achieve in the Requirements Activity?

There are two aims. One aim is to understand as much as possible about the users, their activities, and the context of that activity, so the system under development can support them in achieving their goals. Building on this, our second aim is to produce a set of stable requirements that form a sound basis to start designing. This is not necessarily a major document, nor a set of rigid prescriptions, but it must not change radically in the time it takes to do some design and get feedback on the ideas. Because the end goal is to produce this set of requirements, we shall sometimes refer to this as the requirements activity.

10.2.2 How Can We Achieve This?

At the beginning of the requirements activity, there is a lot to find out and to clarify. At the end of the activity we will have a set of stable requirements that can be the basis of the design activity. In the middle, there are activities concerned with data gathering, analysis,

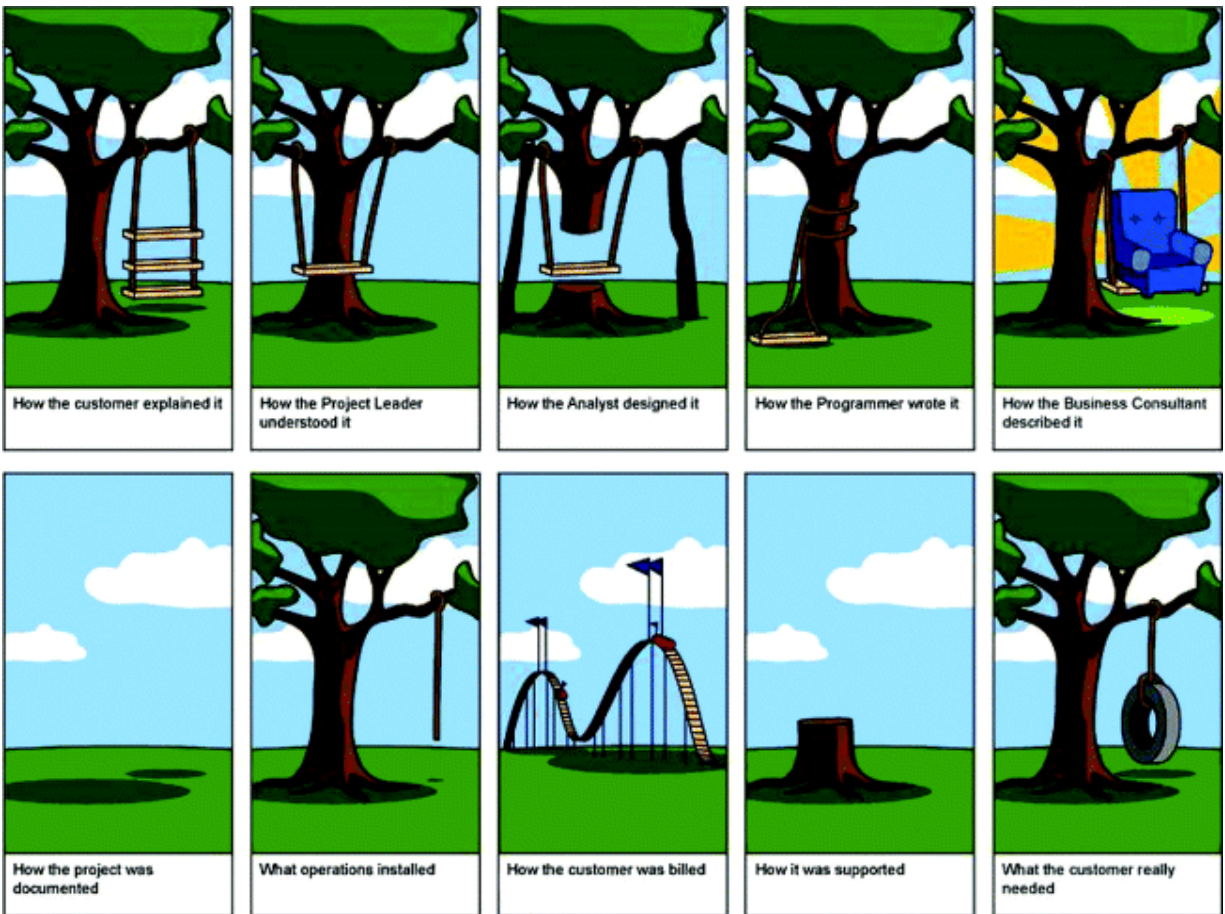
interpretation, and presentation, with the aim of expressing the findings as requirements. Broadly speaking, these activities progress in a sequential manner: first gather some data, then analyze and interpret it, and then extract some requirements from it. But it gets a lot messier than this, and the activities influence one another as the process iterates. Once you start data analysis, you will need to gather more data to clarify or confirm your findings. Also, the way in which requirements are presented may affect your analysis, since it will enable you to identify and express some aspects more easily than others (as discussed in Section 8.7). For example, using a notation which emphasizes the data characteristics of a situation will lead the analysis to focus on this aspect rather than, for example, on task structure. [Chapter 7](#) emphasized that it is valuable to use a complementary set of data gathering and analysis techniques. As we discuss below, there are different kinds of requirements, and each can be emphasized or de-emphasized by different techniques.

Establishing requirements is itself an iterative activity in which the subactivities inform and refine one another. It does not last for a set number of weeks or months and then finish. In practice, requirements evolve and develop as the stakeholders interact with designs and see what is possible and how certain facilities can help them. And as shown in the lifecycle model in [Chapter 9](#), the activity itself will be repeatedly revisited.

10.2.3 Why Bother? The Importance of Getting it Right

Much has been written about the significant cost of fixing errors late in the software development cycle rather than early, during the requirements activity. For example, Davis (1995) identifies insufficient user communication, poor specifications, and insufficient analysis as contributors to poor cost estimation. Boehm and Basili (2001) present a top ten list of software defect reduction findings, the first of which states that “finding and fixing a software problem after delivery is often 100 times more expensive than finding and fixing it during the requirements and design phase.”

The cartoon below illustrates very well what can go wrong if requirements are not clearly articulated.



10.2.4 Why 'Establish' Requirements?

The activity of understanding what a product should do has been given various labels – for example, requirements gathering, requirements capture, requirements elicitation, requirements analysis, and requirements engineering. The first two imply that requirements exist out there and we simply need to pick them up or catch them. Elicitation implies that others (presumably the clients or users) know the requirements and we have to get them to tell us. Requirements, however, are not that easy to identify. You might argue that, in some cases, customers must know what the requirements are because they know the tasks that need supporting. However, they may not have articulated requirements as yet, and even if they have an initial set, they probably have not explored them in sufficient detail for development to begin.

The term requirements analysis is normally used to describe the activity of investigating and analyzing an initial set of requirements that have been gathered, elicited, or captured. Analyzing the information gathered is an important step, since it is this interpretation of the facts, rather than the facts themselves, that inspires the design. Requirements engineering is a better term than the others because it recognizes that developing a set of requirements is an iterative process of evolution and negotiation, and one that needs to be carefully managed and controlled.

We chose the term establishing requirements to represent the fact that requirements have been established from a sound understanding of the users' needs and that they can be justified by and related back to the data collected. Current concerns with cross-cultural design have highlighted the importance of understanding who you are designing for, their specific circumstances, and how to enrich their lifestyles (Chavan *et al*, 2009).

10.3 What Are Requirements?

A requirement is a statement about an intended product that specifies what it should do or how it should perform. One of the aims of the requirements activity is to make the requirements as specific, unambiguous, and clear as possible. For example, a requirement for a smartwatch GPS app might be that the time to load a map is less than half a second. Another, less precise, example might be that teenagers should find the smartwatch appealing. In the case of this latter example, further investigation would be necessary to explore exactly what teenagers would find appealing. Requirements come in many different forms and at many different levels of abstraction, but we need to make sure that the requirements are as clear as possible and that we understand how to tell when they have been fulfilled. The example requirement shown in [Figure 10.1](#) is expressed using a format called an atomic requirements shell from the Volere process (Robertson and Robertson, 2013). This shell requires quite a bit of information about the requirement itself, including something called a fit criterion, which is a way of measuring when the solution meets the

requirement. In [Chapter 9](#) we emphasized the need to establish specific usability criteria for a product early on in development, and this part of the shell encourages this.

Requirement #: 75	Requirement Type: 9	Event/use case #: 6
Description: The product shall issue an alert if a weather station fails to transmit readings.		
Rationale: Failure to transmit readings might indicate that the weather station is faulty and needs maintenance, and that the data used to predict freezing roads may be incomplete.		
Source: Road Engineers		
Fit Criterion: For each weather station the product shall communicate to the user when the recorded number of each type of reading per hour is not within the manufacturer's specified range of the expected number of readings per hour.		
Customer Satisfaction: 3	Customer Dissatisfaction: 5	
Dependencies: None	Conflicts: None	
Supporting Materials: Specification of Rosa Weather Station		
History: Raised by GBS, 28 July 99		

Volere
Copyright © Atlantic Systems Guild

Figure 10.1 An example requirement using the Volere shell

Source: An example requirement using the Volere shell, taken from Robertson and Robertson: “Mastering the Requirements Process” 2013 Addison Wesley. © Atlantic Systems Guild.

10.3.1 Different Kinds of Requirements

In software engineering, two different kinds of requirements have traditionally been identified: functional requirements, which say what the system should do, and non-functional requirements, which say what constraints there are on the system and its development. For example, a functional requirement for a new video game may be that it should be challenging for a range of user abilities. This requirement might then be decomposed into more specific requirements detailing the structure of challenges, e.g. hierarchical levels, hidden tips and tricks, magical objects, and so on. A non-functional requirement for this same game might be that it can run on a variety of platforms such as Xbox, PlayStation, and Wii consoles. A different kind of non-functional requirement would be that it must be delivered in 6 months' time. This represents a constraint on the development activity itself rather than on the product being developed.

Different interactive products will be associated with different constraints. For example, a telecare system designed to monitor an elderly person's movements and alert relevant care staff will be constrained by the type and size of sensors that can be easily worn by the users as they go about their normal activities. Wearable interfaces need to be light, small, fashionable, preferably hidden, and not get in the way. A desirable characteristic of both an online shopping site and a robotic companion is that they should be trustworthy, but this attribute leads to different non-functional requirements: in the former we'd be focused on security of information while in the latter we'd be looking for behavioral norms.

Interaction design involves understanding the functionality required and the constraints under which the product must operate or be developed, hence we are concerned with a wide range of requirements. We will look at some of these in more detail below; [Table 10.1](#) provides an alternative (and long) list of requirements that might be considered.

Table 10.1 The Volere Requirements Specification Template

Source: The Volere Requirements Specification Template version 15 ©Atlantic Systems Guild. The full version of the template may be downloaded from <http://www.volere.co.uk/template.htm>.

PROJECT DRIVERS

1. The Purpose of the Product
2. The Stakeholders

PROJECT CONSTRAINTS

3. Mandated Constraints
4. Naming Conventions and Definitions
5. Relevant Facts and Assumptions

FUNCTIONAL REQUIREMENTS

6. The Scope of the Work
7. Business Data Model and Data Dictionary
8. The Scope of the Product
9. Functional and Data Requirements

NON-FUNCTIONAL REQUIREMENTS

10. Look and Feel Requirements
11. Usability and Humanity Requirements
12. Performance Requirements
13. Operational and Environmental Requirements
14. Maintainability and Support Requirements
15. Security Requirements

- 16. Cultural and Political Requirements
- 17. Legal Requirements

PROJECT ISSUES

- 18. Open Issues
- 19. Off-the-Shelf Solutions
- 20. New Problems
- 21. Tasks
- 22. Migration to the New Product
- 23. Risks
- 24. Costs
- 25. User Documentation and Training
- 26. Waiting Room
- 27. Ideas for Solutions

Functional requirements capture what the product should do. For example, a functional requirement for a robot working in a car assembly plant might be that it should be able to accurately place and weld together the correct pieces of metal. Understanding the functional requirements for an interactive product is fundamental.

Data requirements capture the type, volatility, size/amount, persistence, accuracy, and value of the required data. All interactive products have to handle some data. For example, if the system under consideration is a share-dealing application, then the data must be up-to-date and accurate, and is likely to change many times a day. In the personal banking domain, data must be accurate, must persist over many months and probably years, and there is likely to be a lot of it.

Environmental requirements – or context of use – refer to the circumstances in which the interactive product will operate. Four

aspects of the environment must be considered when establishing requirements. First is the physical environment, such as how much lighting, noise, movement, and dust is expected in the operational environment. Will users need to wear protective clothing, such as large gloves or headgear that might affect the choice of interface type? How crowded is the environment? For example, an ATM operates in a very public physical environment – using a speech interface is therefore likely to be problematic. Box 10.1 illustrates interactive products in a different kind of environment.

The second aspect of the environment is the social environment. The issues raised in [Chapter 4](#) regarding the social aspects of interaction design, such as collaboration and coordination, need to be explored. For example, will data need to be shared? If so, does the sharing have to be synchronous (e.g. does everyone need to be viewing the data at once?) or asynchronous (e.g. two people authoring a report take turns in editing it)? Other factors include the physical location of fellow team members, e.g. do collaborators need to communicate across great distances?

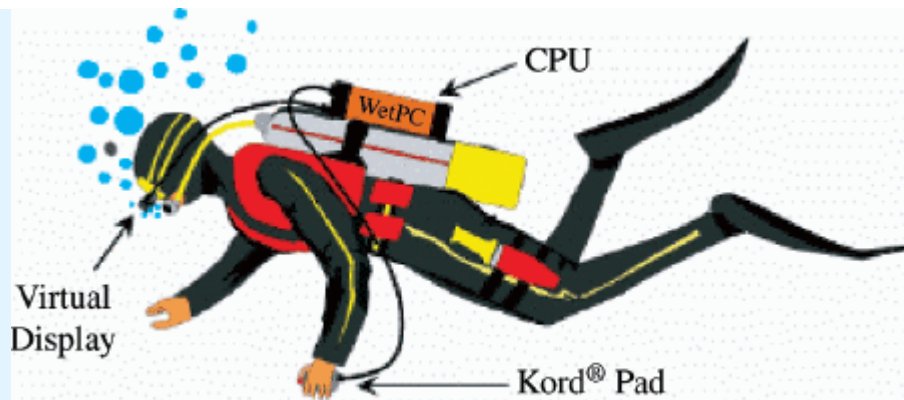
The third aspect is the organizational environment, e.g. how good is user support likely to be, how easily can it be obtained, and are there facilities or resources for training? How efficient or stable is the communications infrastructure? And so on.

Finally, the technical environment will need to be established: for example, what technologies will the product run on or need to be compatible with, and what technological limitations might be relevant?

BOX 10.1

Environmental Requirements: Underwater Computing

Developing a computer for divers to take underwater has one major environmental factor: it is surrounded by water! However, the interface presented more challenges than waterproofing to the designers of WetPC, an underwater wearable computer (see [Figure 10.2a](#)). A miniature personal computer is mounted on the diver's air tank; a mask-mounted head-up display presents the diver with a floating display; and a five-button chordic graphical user interface Kord® Pad is attached to the diver's belt or chest (see [Figure 10.2b](#)).



(a)



(b)

Bottom material description					Particle size
Mud - soft over Gravel - small					< 1mm
Bottom	Mud	Clay	Sand	Gravel	Coral
Smooth	Vegetation			over	
	Shell		Rocks		
No.	Clear				ected
60	OK				

(c)



(d)

Figure 10.2 (a) The components of WetPC's underwater computer. (b) The Kord® Pad chordic keypad. (c) The floating display 'what you see is what you press.' (d) The Kord® Pad in use underwater

Source: Reproduced by permission of WetPC Pty Ltd.
<http://www.wetpc.com.au/WetPC>.

Divers typically have only one hand free to operate the computer, and are likely to be swimming and moving up and down in the water at the same time. So a traditional interface design is no good. The Kord Pad (see [Figure 10.2b](#)) has five keys which can be pressed in different combinations to choose different menu items. It is supported by the floating display interface (see [Figure 10.2c](#)) which tells the divers which keys to press for which actions.

WetPC have also designed SeaSlate which is operated by a six-button keypad called KordGrip (see [Figure 10.3a](#)). SeaSlate can be used to search areas of the sea bottom. Divers can perform operations such as controlling a camera and sending messages. The system is also linked to a GPS device that tells the divers where they are, which facilitates marking the location of mines and other underwater discoveries. □

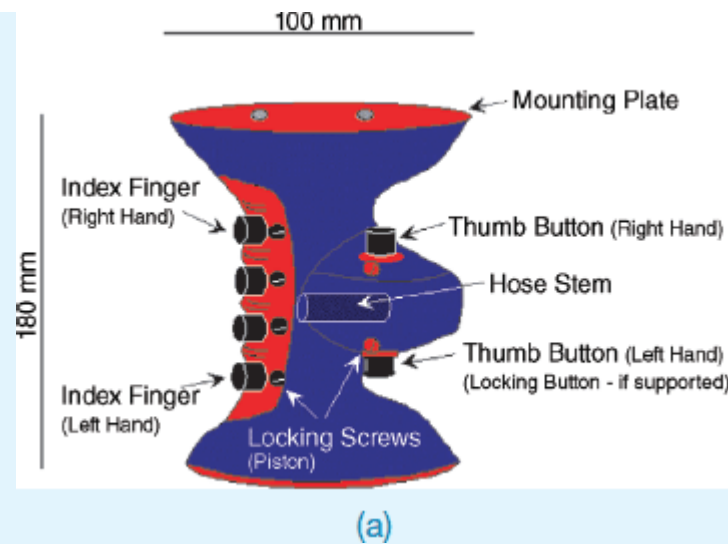


Figure 10.3 (a) The KordGrip interface and (b) the KordGrip in use underwater

Source: (a) Reproduced by permission of WetPC Pty Ltd (b) Reproduced by permission of the Australian Institute of Marine Science.

User characteristics capture the key attributes of the intended user group. In [Chapter 9](#) we mentioned the relevance of a user's abilities and skills, and these are an important aspect of user characteristics. Other characteristics that may affect the design are: the users' nationality, educational background, preferences, personal circumstances, physical or mental disabilities, and so on. In addition, a user may be a novice, an expert, a casual, or a frequent user. This affects the ways in which interaction is designed. For example, a

novice user will require step-by-step instructions, probably with prompting, and a constrained interaction backed up with clear information. An expert, on the other hand, will require a flexible interaction with more wide-ranging powers of control. The collection of characteristics for a typical user is called a user profile. Any one product may have several different user profiles.

In order to bring user profiles to life, they are often transformed into a number of personas (Cooper, 1999). Personas are rich descriptions of typical users of the product under development that the designers can focus on and design the product for. They don't describe real people, but are realistic rather than idealized. Any one persona usually represents a synthesis from a number of real users who have been involved in data gathering. Each persona is characterized by a unique set of goals relating to the particular product under development, rather than a job description or job role. This is because goals often differ between people within the same job role, and people with very different job roles may have the same goals.

As well as goals, a persona will include a description of the user's skills, attitudes, tasks, and environment. These items are all specified in some detail, and so instead of describing someone simply as a competent sailor, they include that he has completed a Day Skipper qualification, has over 100 hours of sailing experience in and around European waters, and gets irritated by other sailors who don't follow the navigation rules. Each persona has a name, often a photograph, and some personal details such as what they do in their leisure time. It is the addition of precise, credible details that helps designers to see the personas as real potential users, and hence as people they can design for.

Usually a product will require a small set of personas rather than just one and it may be helpful to choose one primary persona who represents a large section of the intended user group. Personas are used widely in industry, and have proved to be a powerful way to communicate user characteristics and goals to designers and developers.

The style of personas varies widely. The examples in [Figure 10.4 \(a\)](#) illustrate the persona format developed and used by the company in Box 10.2, and have quite a compact and eye-catching layout.

BOX 10.2

Persona-driven Development in the City of London

Caplin Systems is based in the City of London and provides a framework to investment banks that enables them to quickly build, or enhance, their single-dealer offering, or to create a single-dealer platform for the first time.

The company was drawn to use personas to increase the customer focus of their products by better understanding who they were developing their system for. Personas were seen as a way to provide a unified view of their users, and to start building more customer-focused products.



Figure 10.4 (a) Example personas (b) The narrative journey maps – sad faces show pain points for the persona

Source: Caplin Systems.

The first step was to run a workshop for the whole company, to introduce personas, show how other companies were using them, and for employees to experience the benefits of using personas first hand through some simple team exercises. The proposition was then put forward: 'Should we adopt personas and persona-driven development?' The response was a resounding 'YES!' This was a good thing to do. Gaining this 'buy in' was fundamentally important to ensure everyone was behind the use of personas and committed to the change.

Everyone got excited and work began to define the way forward. Further workshops were run to refine the first persona, though in hindsight the Caplin team think too long was spent trying to get the first persona perfect. Now they are much more agile about persona creation.

Eighteen months after the persona breakthrough workshop, the main persona for Caplin Trader, Jack, and his 'pain points' were the focus of development, design decisions, and team discussions. Ongoing persona development is focusing on end-users of the software built with Caplin's technology, and Narrative Journey Maps capture their interactions and help define goals/motivations and pain points (see [Figure 10.4b](#)). □

Usability goals and user experience goals were described in [Chapter 1](#). These are another kind of requirement, and should be captured together with appropriate measures. [Chapter 9](#) introduced the idea of usability engineering, an approach in which specific measures for the usability goals of the product are agreed upon early in the development process and used to track progress as development proceeds. This both ensures that usability is given due priority and facilitates progress tracking. If we are to follow the philosophy of usability engineering and meet the usability goals, then appropriate requirements need to be identified. The same is true for user experience goals. Although it is harder to identify quantifiable measures that track these qualities, an understanding of their importance should be obtained during the requirements activity.

There are different perspectives that can be taken when identifying measures for usability and user experience goals – some focus on objective measures of the user's performance while others focus on the user's perceptions of the interaction. This difference is discussed further in [Chapters 13–15](#).

Activity 10.1

Suggest some key requirements in each category above (functional, data, environmental, user characteristics, usability goals, and user experience goals) for each of the following situations:

1. An interactive product for use in a university's self-service cafeteria that allows users to pay for their food using a contactless card or smartphone.
2. An interactive product to control the functioning of a nuclear power station.

Comment

Show/Hide

10.4 Data Gathering for Requirements

The overall purpose of data gathering in the requirements activity is to collect sufficient, relevant, and appropriate data so that a set of stable requirements can be produced. Even if a set of initial requirements exists, data gathering will be required to expand, clarify, and confirm those initial requirements. Data gathering needs to cover a wide spectrum of issues: the tasks that users currently perform and their associated goals, the context in which the tasks are performed, and the rationale for the current situation.

You met three common forms of data gathering in [Chapter 7](#): interviews, questionnaires, and observation. Below, we first consider how these three techniques are used in the requirements activity, and then we introduce two other techniques – studying documentation and researching similar products – and their use in establishing requirements. Box 10.3 describes an approach aimed at prompting inspiration rather than simple data gathering.

BOX 10.3

Using Probes for Inspiration

The Presence Project (Gaver *et al*, 1999) looked at novel interaction techniques to increase the presence of elderly people in their local community. The project studied groups in Oslo, Norway, near Amsterdam, the Netherlands, and near Pisa, Italy. Rather than take a more traditional approach of questionnaires, interviews, or ethnographic studies, this project used a novel technique called cultural probes. These probes consisted of a wallet containing a variety of items: eight to ten postcards, about seven maps, a disposable camera, a photo album, and a media diary (see [Figure 10.5](#)). Recipients were asked to answer questions associated with certain items in the wallet, then to return them directly to the researchers. For example, the postcards had pictures on the front and questions on the back, and were pre-addressed and stamped so that they could be easily returned. Questions included 'Please tell us a piece of advice or insight that has been important to you,' 'What place does art have in your life?' and 'Tell us about your favorite device.' The maps and associated inquiries were designed to find out about the participants' attitudes towards their environment. They were printed on various textured papers and were in the form of folding envelopes, also to facilitate their return. On local maps, participants were asked to mark sites where they would go to meet people, to be alone, to daydream, and where they would like to go, but couldn't. On a map of the world, they were asked to mark places where they had been.



Figure 10.5 A cultural probe package

Source: B. Gaver, T. Dunne and E. Pacenti (1999): “Cultural Probes” from *Interactions* 6(1) pp.21–29. ©1999 Association for Computing Machinery, Inc. Reprinted by permission.

Participants were asked to use the camera to take pictures of their home, what they will wear today (whenever ‘today’ was), the first person they see today, something desirable, and something boring. In the photo album they were asked to tell the researchers their story in pictures. The media diary was to record their use of television and radio.

“What we learned about the elders is only half the story, however. The other half is what the elders learned from the probes. They provoked the groups to think about the roles they play and the pleasures they experience, hinting to them that our designs might suggest new roles and new experiences” (Gaver *et al*, 1999, p. 29).

The probe idea has been adapted and adopted since its beginnings for various settings (Boehner *et al*, 2007, Wallace *et al*, 2013). Extending this idea to toolkits and technology probes, such as Lego Mindstorms or the SenseBoard (Richards and Woodthorpe, 2009), users themselves have been inspired to directly develop innovative and unexpected products (Rogers *et al*, 2014). □

Interviews.

Interviews are good for exploring issues, and semi-structured or unstructured interviews are often used early on to elicit scenarios (see Section 10.6.1). In the context of establishing requirements, it is equally important for development team members to meet stakeholders and for users to feel involved. This on its own may be sufficient motivation to arrange interviews.

Focus groups.

Focus groups are good for gaining a consensus view and highlighting areas of conflict and disagreement during the requirements activity. On a social level it also helps for stakeholders to meet designers and each other, and to express their views in public. It is not uncommon for one set of stakeholders to be unaware that their understanding of an issue or a process is different from another's, even though they are in the same organization.

The generic idea of a focus group has been tailored for use within the requirements activity and requirements workshops have grown in popularity. Each workshop is carefully planned, attendees are carefully chosen, and specific deliverables are produced. Gottesdiener (2002) suggests a very useful, practical approach to

requirements workshops that emphasizes planning and deliverables but also collaboration and facilitation.

Questionnaires.

Questionnaires may be used for getting initial responses that can then be analyzed to choose people to interview or to get a wider perspective on particular issues that have arisen elsewhere. For example, a questionnaire might be used in order to gauge whether a new university online help service would be welcomed by students. This questionnaire could ask for impressions and opinions about current support services and whether the respondent is prepared to be interviewed further. Or the questionnaire might be used to get opinions and views about specific suggestions for the kind of help that would be most appreciated.

Direct observation.

In the requirements activity, observation of participants in their natural setting is used to understand the nature of the tasks and the context in which they are performed. Sometimes the observation is carried out by trained observers who record their findings and report them back to the design team, and sometimes the observation is carried out by or with a member of the design team.

Indirect observation.

Diaries and interaction logging are used less often within the requirements activity where a new product is under development. However, if a product is being developed iteratively or is evolving from an existing product, indirect observation and experience sampling are very valuable. Interaction logging together with sophisticated web analytics are particularly useful for improving websites.

Studying documentation.

Manuals and other documentation are a good source of data about the steps involved in an activity and any regulations governing a task. Documentation should not be used as the only source,

however, as everyday practices may augment it. Taking a user-centered view of development means being interested in everyday practices rather than an idealized account.

Studying documentation is good for understanding legislation and getting some background information on the work. It also doesn't involve stakeholder time, which is a limiting factor on the other techniques.

Researching similar products.

In [Chapter 9](#) we talk about looking at similar products in order to generate alternative designs. Another reason to look at similar products is to help prompt requirements. For example, when developing an image editor for a mobile device, Kangas and Kinnunen (2005) report that they looked at PC image editing software in order to gain an understanding of the kinds of features and interaction that such a package might offer. Similarly, Chisnell and Brown (2004) evaluated competitors' health plan websites when redeveloping their own.

The choice of data gathering techniques for the requirements activity is influenced by several factors including the nature of the task, the participants, the analyst, and the resources available (see [Chapter 7](#) for a more detailed discussion of these factors). It is usual for more than one data gathering technique to be used in order to provide different perspectives. For example, observation to understand the context of task performance, interviews to target specific user groups, questionnaires to reach a wider population, and focus groups to build a consensus view. There is no right choice or combination as this will depend on the specific circumstances and resources. Many different combinations are used in practice. Box 10.4 includes some examples of the different combinations used. Note that the example from Shaer *et al* (2012) also illustrates the development of an interactive product for a specialist domain, where users join the development team to help them understand the domain complexities. Also note that evaluating prototypes may be included as part of the requirements activity. This serves to highlight the close relationship between requirements, design, and evaluation,

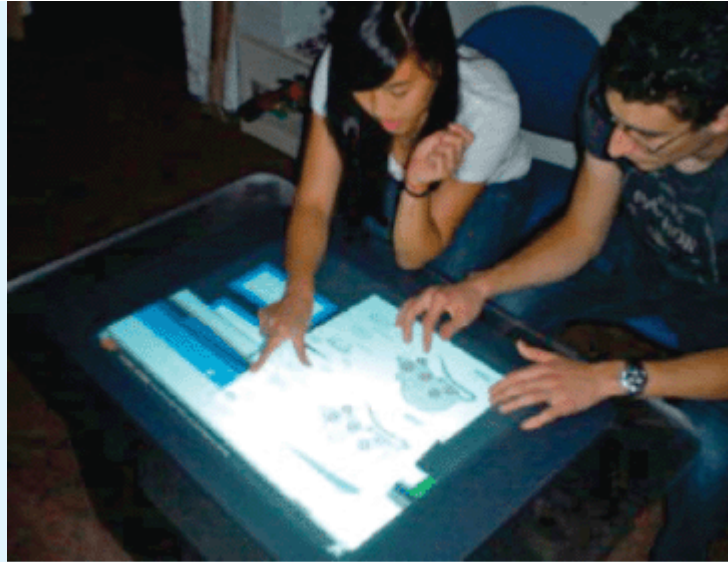
as illustrated in our interaction design process in [Chapter 9](#) – the distinction between these phases is blurred and depends mainly on emphasis.

BOX 10.4

Combining Data Gathering in Requirements Activities

Diary and interviews: Dearman *et al* (2008) were investigating the types of information people need and share in their everyday lives. They conducted a four-week diary study with 20 participants. Each participant was given a paper diary with predefined forms and was asked to complete a form every time they needed some information or shared information. Participants were sent a text message (or email message) twice a day to remind them to keep these entries up to date, and were interviewed each week to elicit further details, hand over the diary for the week, and collect a new diary for the next week. At the end of the study exit interviews were conducted with each participant to understand their overall experience.

Ethnographic study, interviews, usability tests, and user participation: Shaer *et al* (2012) report on the design of a multi-touch tabletop user interface for collaborative exploration of genomic data (see [Figure 10.6](#)). In-depth interviews were conducted with 38 molecular and computational biologists to understand the current work practices, needs, and workflow of small research groups. A small team of nine researchers investigating gene interaction in Tuberculosis was studied for eight weeks using an ethnographic approach, and other labs were also observed. Because the application area was specialized, the design team needed to be comfortable with the domain concepts. To achieve this, biologists were integrated into the development team, and other members of the design team regularly visited biology research group partners, attended courses to teach them relevant domain concepts, and had frequent usability tests with users.



(a)



(b)

Figure 10.6 (a) Exploring mouse gene expression using G-nome Surfer 2.0 (b) G-nome Surfer Pro displaying the chromosome visualizations, an aligned sequence, and publications

Source: Shaer *et al* (2012) The design, development, and deployment of a tabletop interface for collaborative exploration of genomic data, *International Journal of Human–Computer Interaction* 70, 746–764. ©2012 Association for Computing Machinery, Inc. Reprinted by permission.

The result of this two-year effort was a set of five main requirements which were used as the basis of development:

alleviating data explosion; lowering the threshold for using advanced bioinformatics tools; facilitating an integrated and flexible workflow; supporting multiple forms of evidence; and fostering collaboration and reflection.

Documentation, interview, online survey, group discussion:

Oostveen and van den Besselaar (2004) describe the development of a smart-card-based system that would support mobile European citizens in achieving various administrative transactions between countries. For example, moving from one country to another requires a set of bureaucratic operations to be performed, largely concerned with exchanging documentation. This system would enable the citizen to download the required information onto a smart card and use this electronic document in the new country. In-depth interviews with expatriates from different countries allowed them to catalog the problems mobile Europeans encounter. Interviews with civil servants and various intermediaries whose jobs involve helping such people were also held. In order to check the relevance of the interview results to a wider set of potential users, they then set up an online survey. The administrative processes were investigated through group discussions which were held in different countries between technology designers and administrative experts; potential solutions were also discussed in these workshops. Documentation was studied to underpin the other requirements activities.

Focus groups, interviews, and evaluations: Liu *et al* (2005) report on some research into expanding the use of ATMs in China. This study was conducted in order to identify factors that might affect requirements for ATM design, identify problems or issues with current ATMs in China, and identify appropriate ways in which to increase adoption. They used three different approaches to data gathering: focus groups were used to share good and bad experiences of ATM use, interviews were carried out in a shopping center with 100 participants to find out how widely applicable the findings were from the focus groups, and

evaluations of existing ATMs were conducted to uncover confusions and error rates. □

10.4.1 Contextual Inquiry

Contextual inquiry (Holtzblatt and Jones, 1993) is one of seven parts of contextual design (Beyer and Holtzblatt, 1998), which is a structured approach to the collection and interpretation of data from fieldwork with the intention of building a software-based product. Contextual design involves the production of five different models of work.

Contextual inquiry is a popular technique for uncovering requirements, and in particular in uncovering requirements relating to the context of use. For example Meschtscherjakov *et al* (2011) used contextual inquiry over a period of six weeks to investigate how car drivers interact with the multi-function rotating button often found in the centre of the car dashboard. The study highlighted that comfort, safety, and security factors were mentioned most often, while visual and haptic aesthetics were not mentioned by any participants. These findings informed the design of new in-vehicle information systems.

Contextual inquiry is an approach that emerged from the ethnographic approach to data gathering. It is tailored to gather data that can be used in design and it follows an apprenticeship model: the designer works as an apprentice to the user. The most typical format for contextual inquiry is a contextual interview, which is a combination of observation, discussion, and reconstruction of past events. Contextual inquiry rests on four main principles: context, partnership, interpretation, and focus.

The context principle emphasizes the importance of going to the workplace and seeing what happens. The partnership principle states that the developer and the user should collaborate in understanding the work; in a traditional interviewing or workshop situation, the interviewer or workshop leader is in control, but in contextual inquiry the spirit of partnership means that the understanding is developed through cooperation.

The interpretation principle says that the observations must be interpreted in order to be used in design, and this interpretation should also be developed in cooperation between the user and the developer. For example, I have a set of paper notes stuck above my desk at work; some list telephone numbers and some list commands for the software I use. Someone coming into my office might interpret these facts in a number of ways: that I don't have access to a staff directory; that I don't have a user manual for my software; that I use the software infrequently; that the commands are particularly difficult to remember. The best way to interpret these observations is to discuss them with me. In fact, I do have access to a staff directory, but I would need to log on to a different network than my usual one, which is distracting and takes time. The commands are there because I often forget them and waste time searching through menu structures.

The fourth principle, the focus principle, is related to our discussions in [Chapter 7](#) about keeping the data gathering focused on your goals. In contextual inquiry, as in an unstructured interview, for example, it is easy for the discussion to wander off target. To help avoid this, a project focus is established to guide the interviewer, which will then be augmented by the individual's own focus that arises from their perspective and background.

Normally, each member of the team developing the interactive product conducts at least one contextual inquiry session. Data is collected in the form of notes and perhaps audio and video recording, but a lot of information is in the observer's head. It is important to review the experience and to start documenting the findings as soon as possible after the session. Contextual inquiry is usually followed by an interpretation session in which a number of models are generated: an affinity diagram, the work flow model, the sequence model, the artifact model, the cultural model, and the physical model. More detail about these models and how to generate them is in Beyer and Holtzblatt (1998).

Contextual inquiry has been adapted for use in a remote setting (Rampoldi-Hnilo and English, 2004) and contextual design has been

adapted for use with a shortened product cycle (Holtzblatt *et al*, 2004).

Activity 10.2

How does the contextual inquiry interview compare with the interviews introduced in [Chapter 7](#)?

Comment

Show/Hide

10.4.2 Data Gathering Guidelines for Requirements

General advice for data gathering is given in [Chapter 7](#), but here are a few more specific guidelines worthy of note when gathering data for requirements:

- Focus on identifying the stakeholders' needs. This may be achieved by studying their existing behavior and support tools, or by looking at other products, such as a competitor's product or an earlier release of your product under development.
- Involve all the stakeholder groups. It is very important to make sure that you get the views of all the right people. This may seem an obvious comment, but it is easy to overlook certain sections of the stakeholder population if you're not careful. We were told about one case where a large distribution and logistics company reimplemented their software systems and were very careful to involve all the clerical, managerial, and warehouse staff in their development process, but on the day the system went live, the productivity of the operation fell by 50%. On investigation it was found that the bottleneck was not in their own company, but in the suppliers' warehouses that had to interact with the new system. No one had asked them how they worked, and the new system was incompatible with their working routines.

- Involving only one representative from each stakeholder group is not enough, especially if the group is large. Everyone you involve in data gathering will have their own perspective on the situation, the task, their job, and how others interact with them. If you only involve one representative stakeholder then you will only get a narrow view.
- Support the data gathering sessions with suitable props, such as task descriptions and prototypes if available. Since the requirements activity is iterative, prototypes or descriptions generated during one session may be reused or revisited in another with the same or a different set of stakeholders. Using props will help to jog people's memories and act as a focus for discussions. Maiden *et al* (2007b) recommend the use of mobile technologies and dedicated requirements applications to support the requirements activity.

10.5 Data Analysis, Interpretation, and Presentation

Methods and approaches for analysis, interpretation, and presentation of data are discussed in [Chapter 8](#). These are applicable during the requirements activity to structure and record descriptions of requirements. Using a format such as the Volere shell ([Figure 10.7](#)) highlights the kinds of information to look for and is a good first step in data analysis for requirements. Note that many of the entries are concerned with traceability. For example, who raised the requirement and where can more information about it be found? This information may be captured in documents or in diagrams drawn during analysis.

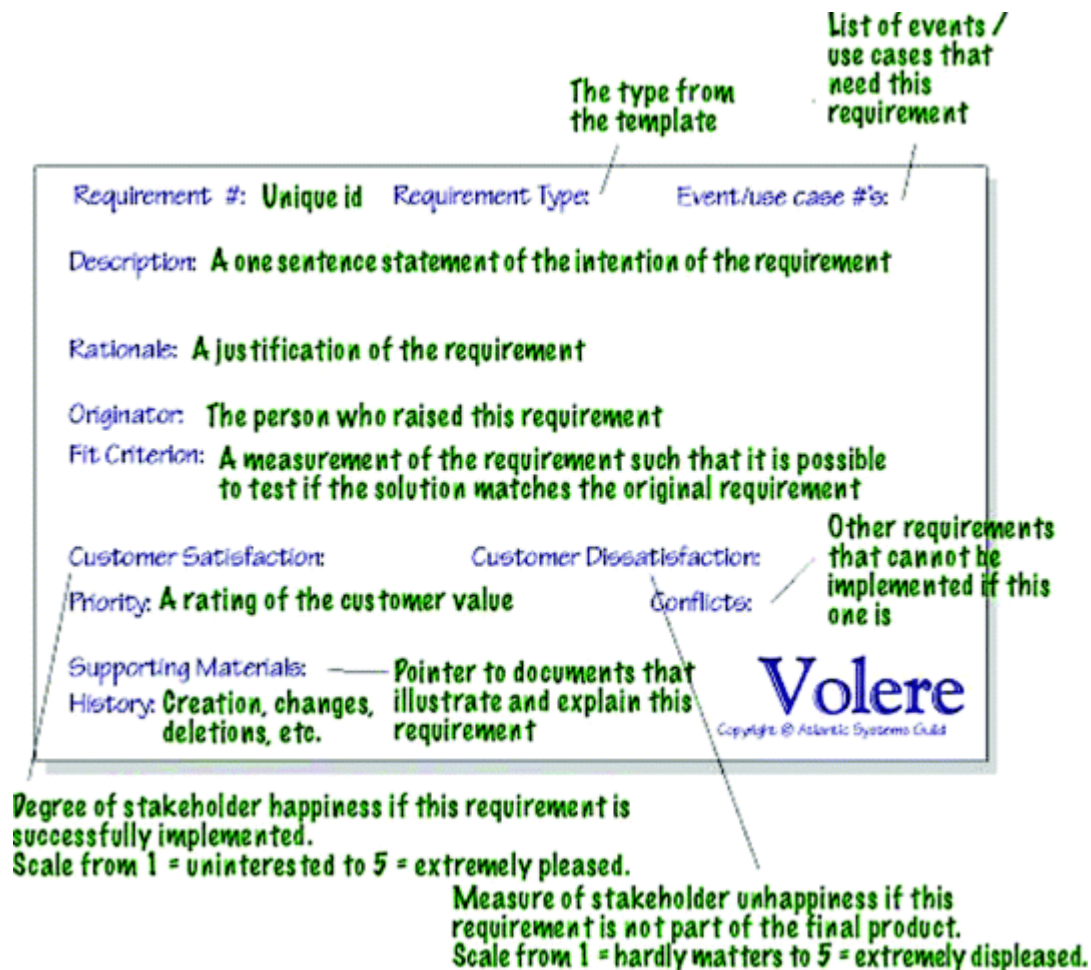


Figure 10.7 The Volere shell for requirements

Source: © Atlantic Systems Guild.

Some kinds of requirements are best investigated using more formal techniques and notations. For example, functional requirements may be analyzed and documented using class diagrams, state charts, and sequence diagrams, among others. Data requirements can be expressed using entity-relationship diagrams, and so on. Examples of two such diagrams representing a portion of a flight booking system are given in [Figure 10.8](#). These diagrams can be linked to the requirements through the Event/use case field in the shell in [Figure 10.7](#).

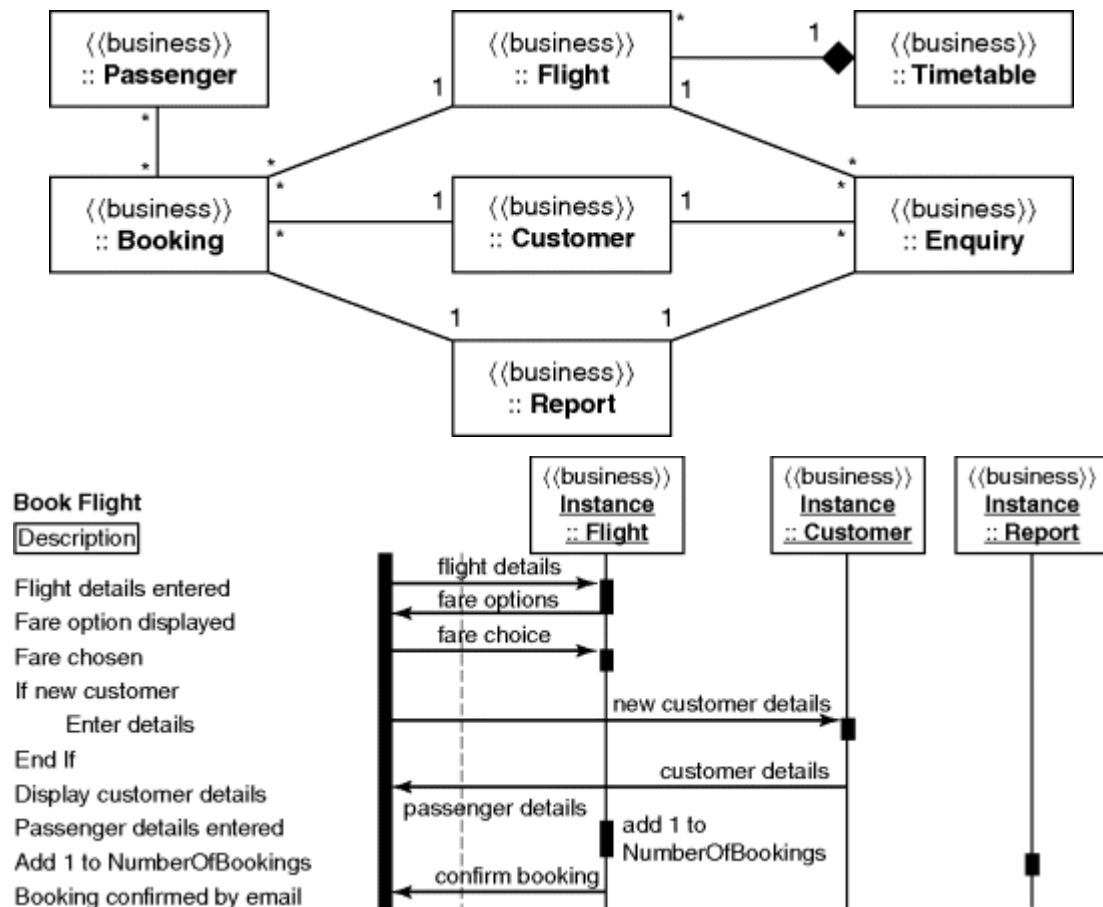


Figure 10.8 (a) Class diagram and (b) sequence diagram that might be used to analyze and capture static structure and dynamic behavior (respectively)

We don't go into the detail of how specialized diagrams such as these might be developed, as whole books are dedicated to them. Instead, we describe four techniques that have a user-centered focus and are used to understand users' goals and tasks: scenarios (Section 10.6.1), use cases (Section 10.6.2), essential use cases (or task cases) (Section 10.6.3), and task analysis (Section 10.7). All of them may be produced as a result of a data gathering session, and their output used as props in subsequent data gathering sessions.

The requirements activity is iterated a number of times before a stable set of requirements evolves. As more analysis techniques are applied, a deeper understanding of requirements will emerge and the requirements descriptions will expand and clarify.

10.5.1 Brainstorming for Innovation

So far we have focused on how requirements may emerge directly from the data gathered. However, establishing a suitable set of requirements is likely to also involve innovation. Brainstorming is not a technique specific to interaction design, or to any other discipline, and is a generic technique used to generate, refine, and develop ideas. It is widely used in interaction design specifically for generating alternative designs (as discussed in [Chapter 9](#)) or for suggesting new and better ideas for supporting users.

Various rules have been suggested for making a brainstorming session successful, some of which we list below. For requirements, two key success factors are firstly that the participants should know the users' goals that the product is to support, and secondly that no ideas should be criticized or debated (Robertson and Robertson, 2013; Kelley, 2008). Some other suggestions for successful requirements brainstorms are:

1. Include participants from a wide range of disciplines, with a broad range of experience (Robertson and Robertson, 2013; Kelley, 2008).
2. Don't ban silly stuff (Kelley, 2008). Wild ideas often turn into really useful requirements (Robertson and Robertson, 2013).
3. Use catalysts for further inspiration. Build one idea on top of another (Robertson and Robertson, 2013). Kelley (2008) also suggests jumping back to an earlier idea, or considering alternative interpretations when energy levels start to flag. If you get stuck, use a word pulled randomly from a dictionary to prompt ideas related to the product (Robertson and Robertson, 2013).
4. Keep records. Robertson and Robertson (2013) suggest that every idea should be captured, without censoring. Kelley (2008) suggests that you number them so that you can refer back to ideas more easily. He also suggests that the walls and tables in the room be covered in paper and that participants be encouraged to sketch, mind-map, and diagram ideas, including

keeping the flow of ideas, as spatial memory is very strong and this can facilitate recall.

5. Sharpen the focus (Kelley, 2008). Start the brainstorm with a well-honed problem. This will get the brainstorm off to a good start and makes it easier to pull people back to the main topic if the session wanders.
6. Use warm-up exercises and make the session fun (Robertson and Robertson, 2013). The group will require warming up if they haven't worked together before, most of the group don't brainstorm regularly, or the group is distracted by other pressures (Kelley, 2008). Warm-up exercises might take the form of word games, or the exploration of physical items related or unrelated with the problem at hand. For example, see the description of the TechBox in [Chapter 9](#).

10.6 Task Description

Descriptions of business tasks have been used within software development for many years. During the 1970s and 1980s, business scenarios were commonly used as the basis for acceptance testing, i.e. the last testing stage before the customer paid the final fee installment and accepted the system. In more recent years, due to the emphasis on involving users earlier in the development lifecycle and the large number of new interactive products now being developed, task descriptions are used throughout development, from early requirements activities through prototyping, evaluation, and testing. Consequently, more time and effort has been put into understanding how best to structure and use them.

As shown by Alexander and Maiden's (2004) collection of scenarios, stories, and use cases, there are many different flavors of task description, and they can be used for different purposes, emphasizing different elements of the product being developed. For example, Alexander and Maiden use a structuring framework that distinguishes task descriptions according to four views which are made up of nine facets including method of description (e.g. text,

graphics, image or prototype, and formal, informal, or semi-formal notation), context (e.g. organizational environment and system interaction), and role (descriptive, exploratory, or explanatory).

We shall introduce three of the more common description types here: scenarios, use cases, and essential use cases (sometimes referred to as task cases). Each of these may be used to describe either existing tasks or envisioned tasks with a new product. They are not mutually exclusive and are often used in combination to capture different perspectives or to document different stages during the development lifecycle.

In this section and the next, we use two main examples to illustrate the application of techniques. These are a movie rental subscription service and a shared travel organizer. The movie rental service allows subscribers to rent movies of their choice; the shared travel organizer supports a group of people who are exploring vacation possibilities.

10.6.1 Scenarios

A scenario is an ‘informal narrative description’ (Carroll, 2000). It describes human activities or tasks in a story that allows exploration and discussion of contexts, needs, and requirements. It does not necessarily describe the use of software or other technological support to achieve a task. Using the vocabulary and phrasing of users means that scenarios can be understood by the stakeholders, and they are able to participate fully in the development process. In fact, the construction of scenarios by stakeholders is often the first step in establishing requirements.

Imagine that you have just been invited along to talk to a group of users who perform data entry for a university admissions office. You walk in, and are greeted by Sandy, the supervisor, who starts by saying something like:

Well, this is where we process admissions forms. We receive about 50 a day during the peak application period. Brian here checks the applications to see that they are complete, that is, that all the information and supporting evidence has been included. You see, we require evidence of relevant school exam results or work experience before we can process the application. Depending on the result of this initial inspection, the applications are sent on to . . .

Telling stories is a natural way for people to explain what they are doing or how to achieve something. It is therefore something that stakeholders can easily relate to. The focus of such stories is also naturally likely to be about what the users are trying to achieve, i.e. their goals. Understanding why people do things as they do and what they are trying to achieve in the process allows us to concentrate on the human activity rather than interaction with technology.

This is not to preserve existing behavior in the new product, but to understand current behavior and explore the constraints, contexts, irritations, facilitators, and so on under which people operate. It also allows us to identify the stakeholders and the artifacts involved in an activity. Repeated reference to a particular app, book, behavior, or location indicates that this is somehow central to the activity being performed and that we should take care to understand what it is and the role it plays.

A scenario that might be generated by potential users of an online movie rental subscription service is given below:

Say I want to find a movie directed by Martin Scorsese. I don't remember the title but I know it came out in the cinemas around 2006 or 2007. I go to the service website and choose the director option. A huge list of directors is displayed – I had no idea there were so many directors with surnames beginning with S! After scrolling through the list I find Martin Scorsese and choose to see further details about him. Another long list of movies eventually leads me to the movie I was looking for – The Departed. As an existing subscriber, I need to log in to be able to rent the movie. Once my login has been confirmed, I can choose the rental period and payment method. I have my preferences already registered in the system, so I just choose the defaults and download my movie.

In this limited scenario of existing system use, there are some things of note: the long lists of names and movies that the user has to scroll through, the lack of detailed search possibilities, the importance of choice around rental period, and the usefulness of having default settings chosen by regular users. These are all indicators of potential design choices for the new system. The scenario also tells us one (possibly common) use of the system: to search for a movie by a specific director when we don't know the title.

The level of detail present in a scenario varies depending on where in the development process they are being used. During requirements it is a good idea for scenarios to emphasize the context, the usability and user experience goals, and the tasks the user is performing. When used in combination with detailed personas, this kind of scenario can improve the developers' appreciation of the user experience.

Often scenarios are generated during workshop, interview, or brainstorming sessions to help explain or discuss some aspect of the user's goals. They can be used to imagine potential uses of a product as well as to capture existing behavior. They are not intended to capture a full set of requirements, but are a very personalized account, offering only one perspective.

The following scenario for the shared travel organizer was elicited in an informal interview. This describes how one function of the system might work: to identify potential vacation options. Note that this scenario includes details about some typical users and their needs. This is the kind of information that you might glean from a requirements interview.

The Thomson family enjoy outdoor activities and want to try their hand at sailing this year. There are four family members: Sky (10 years old), Eamonn (15 years old), Claire (35), and Will (40). One evening after dinner they decide to start exploring the possibilities. They all gather around the travel organizer and enter their initial set of requirements – a sailing trip for four novices in the Mediterranean. The console is designed so that all members of the family can interact easily and comfortably with it. The system's initial suggestion is a flotilla, where several crews (with various levels of experience) sail together on separate boats. Sky and Eamonn aren't very happy at the idea of going on vacation with a group of other people, even though the Thomsons would have their own boat. The travel organizer shows them descriptions of flotillas from other children their ages and they are all very positive, so eventually, everyone agrees to explore flotilla opportunities. Will confirms this recommendation and asks for detailed options. As it's getting late, he asks for the details to be saved so everyone can consider them tomorrow. The travel organizer emails them a summary of the different options available.

Scenarios may also be constructed to describe an envisioned situation in the future. An example of a futuristic scenario showing how the skin can be used for input is shown below and the technology is illustrated in [Figure 10.9](#). The technology for such input has been developed and tested (Harrison *et al*, 2010), and the scenario illustrates how it may be used commercially.

Bramat has just finished his daily 4 mile run. He likes listening to music while he exercises, and has been playing his favorite pieces. This new skinput technology is great as he can focus on the running while scrolling through the available tracks, skipping through them with a simple tap of his fingers. He comes in exhausted and flops down on his favorite seat. With a flick of his fingers he turns off his music player and opens the palm of his hand to reveal the television remote control panel, graphically projected on his skin. He taps on a button to choose the station for the program he wants, adjusts the volume with a few more taps, and sits back to watch. Feeling hungry, he walks to his kitchen, opens his palm once again and sees a list of recipes possible given the items in his fridge. With another hand gesture, his palm turns into a telephone keypad, from where he can invite a friend over for dinner.

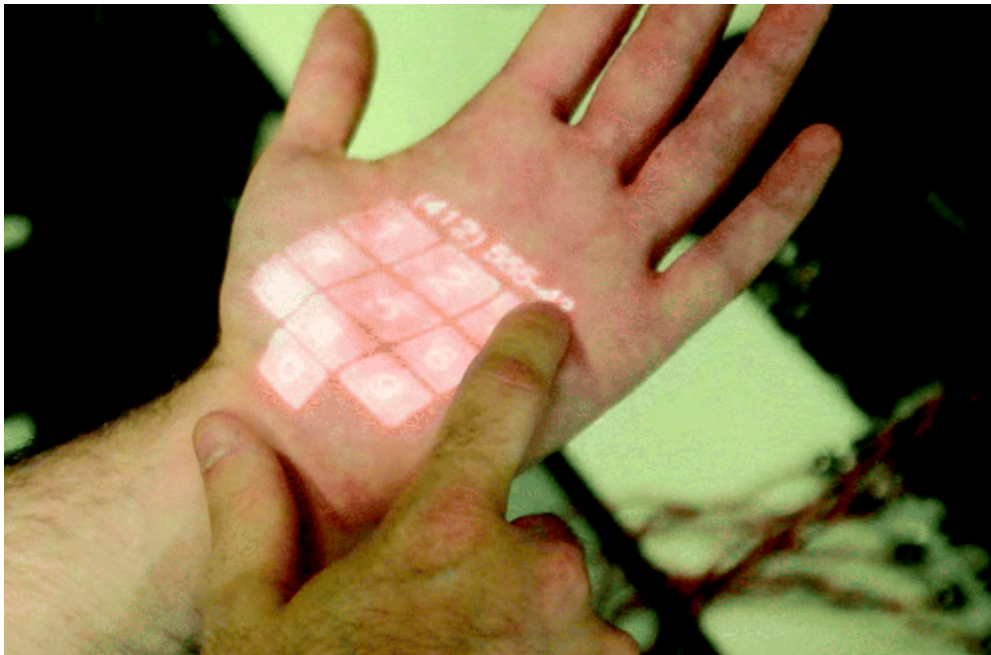


Figure 10.9 How skinput might be used

Source: Reproduced by permission of Chris Harrison, Dan Morris, Desney Tan - Microsoft Research & Carnegie Mellon University.

Video about skinput at <http://youtu.be/g3XPudW9Ryg>

In this chapter, we refer to scenarios only in their role of helping to establish requirements. They have a continuing role in the design process that we shall return to in [Chapter 11](#). Indeed, as Alexander and Maiden (2004) show, scenarios have a role to play throughout the lifecycle, and Rosson and Carroll (2002) explain an approach called scenario-based usability engineering that illustrates the use of scenarios within a usability engineering framework.

Capturing scenarios of existing behavior and goals helps in determining new scenarios and hence in gathering data useful for establishing the new requirements. The next activity is intended to help you appreciate how a scenario of existing activity can help identify the requirements for a future application to support the same user goal.

Activity 10.3

Write a scenario of how you would currently go about choosing a new car. This should be a brand new car, not a second-hand car. Having written it, think about the important aspects of the task: your priorities and preferences. Then imagine a new interactive product that supports you in your goal and takes account of these issues. Write a futuristic scenario showing how this product would support you.

Comment

Show/Hide

BOX 10.5

Scenarios and Personas

The difference between personas and scenarios can be difficult to grasp at first. Although reading about it may seem clear, it is a common confusion to end up combining the two. A scenario describes one use of a product or one example of achieving a goal, while a persona characterizes a typical user of the product. [Figure 10.10](#) captures this difference graphically.

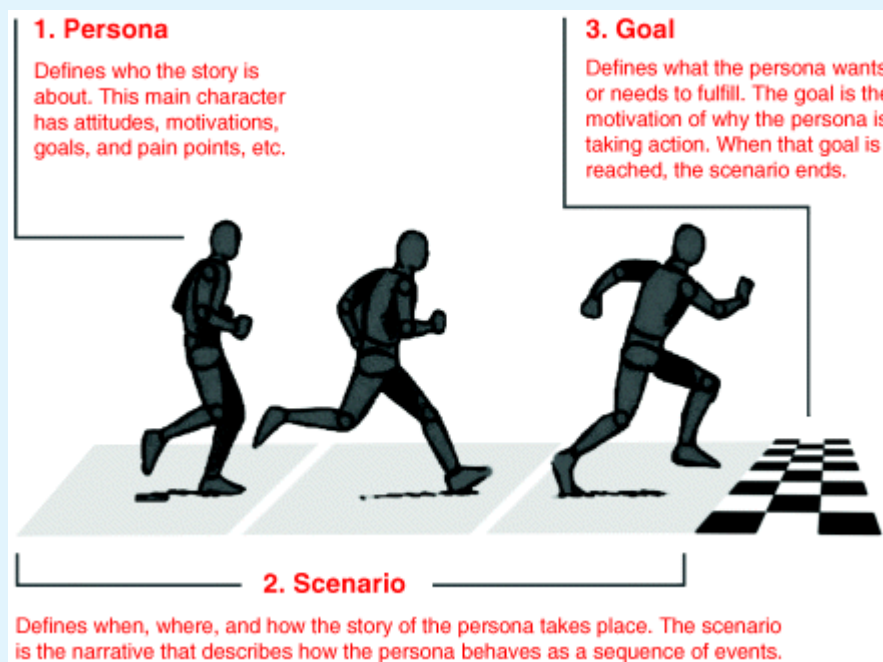


Figure 10.10 The relationship between a scenario and its associated persona

Source: <http://www.smashingmagazine.com/2014/08/06/a-closer-look-at-personas-part-1/>

Note that this figure introduces the notion of a scenario goal. Thinking about the persona's goal for the scenario helps to scope the scenario to one use of the product. □

10.6.2 Use Cases

Use cases also focus on user goals, but the emphasis here is on a user–system interaction rather than the user's task itself. They were originally introduced through the object-oriented community in the book *Object-Oriented Software Engineering* (Jacobson *et al*, 1992). Although their focus is specifically on the interaction between the user (called an actor) and a software system, the stress is still very much on the user's perspective, not the system's. The term scenario is also used in the context of use cases. In this context, it represents one path through the use case, i.e. one particular set of conditions. This meaning is consistent with the definition given above in that they both represent one specific example of behavior.

A use case is associated with an actor, and it is the actor's goal in using the system that the use case wants to capture. In this technique, the main use case describes what is called the normal course, i.e. the set of actions that the analyst believes to be most commonly performed. So, for example, if through data gathering we have found that most subscribers to the movie rental service know the title of the movie they want to rent, then the normal course for the use case would include the steps necessary to find the movie by title. Other possible sequences, called alternative courses, are then listed at the bottom of the use case.

A use case for retrieving the visa requirements using the travel organizer, with the normal course being that information about the visa requirements is available, might be:

1. The system displays options for investigating visa and vaccination requirements.
2. The user chooses the option to find out about visa requirements.
3. The system prompts the user for the name of the destination country.
4. The user enters the country's name.
5. The system checks that the country is valid.
6. The system prompts the user for her nationality.

7. The user enters her nationality.
8. The system checks the visa requirements of the entered country for a passport holder of her nationality.
9. The system displays the visa requirements.
10. The system displays the option to print out the visa requirements.
11. The user chooses to print the requirements.

Alternative courses:

6. If the country name is invalid:
 - 6.1 The system displays an error message.
 - 6.2 The system returns to step 3.
8. If the nationality is invalid:
 - 8.1 The system displays an error message.
 - 8.2 The system returns to step 6.
9. If no information about visa requirements is found:
 - 9.1 The system displays a suitable message.
 - 9.2 The system returns to step 1.

Note that the number associated with the alternative course indicates the step in the normal course that is replaced by this action or set of actions. Also note how specific the use case is about how the user and the system will interact.

Use cases may be described graphically. [Figure 10.11](#) shows the use case diagram for the travel organizer. The Travel agent actor is associated with the use case 'Update travel details.' Another actor for the travel organizer is Traveler, such as the Thomson family. Actors may be associated with more than one use case, so, for example, Traveler is associated with a use case 'Identify potential vacations' as well as the 'Retrieve visa requirements' use case. Each use case may also be associated with more than one actor. Note that an actor represents a role, so when Jasmine, who works for the travel

agency, is booking a trip for herself, she adopts the role of the Traveler actor, but when she is working for the travel agent she will adopt the role of Travel agent.

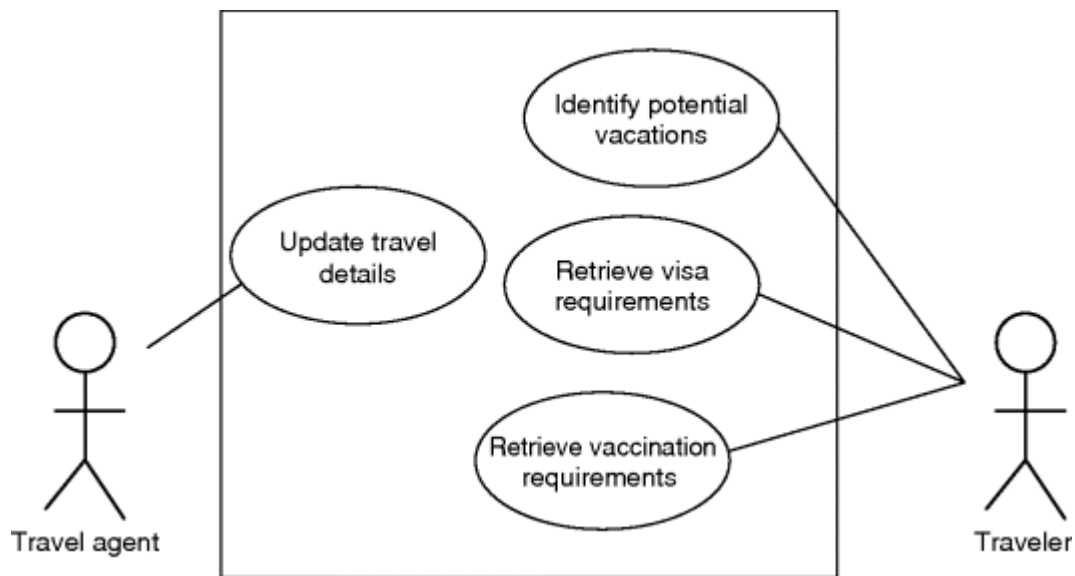


Figure 10.11 Use case diagram for the travel organizer showing four use cases and two actors

This kind of description has a different style and a different focus from the scenarios described in Section 10.6.1. The layout is more formal, and the structure of good use cases has been discussed by many (e.g. Bittner and Spence, 2002; Alexander and Maiden, 2004). The description also focuses on the user–system interaction rather than on the user's activities; thus a use case presupposes that technology is being used. This kind of detail is more useful at conceptual design stage than during requirements or data gathering, but use cases have been found to help some stakeholders express their views on how existing systems are used and how a new system might work.

To develop a use case, first identify the actors, i.e. the people or other systems that will be interacting with the system under development. Then examine these actors and identify their goal or goals in using the system. Each of these will be a use case.

Activity 10.4

Consider the example of the movie rental service. One use case is 'Rent movie,' and this would be associated with the Subscriber actor.

1. Identify one other main actor and an associated use case, and draw a use case diagram for the movie rental service.
2. Write out the use case for 'Rent movie' including the normal and some alternative courses. You may assume that the normal course is for users to go to the website to find a movie by director.

Comment

Show/Hide

10.6.3 Essential Use Cases

Essential use cases were developed by Constantine and Lockwood (1999) to combat what they see as the limitations of both scenarios and use cases as described above. Scenarios are concrete stories that concentrate on realistic and specific activities. They therefore can obscure broader issues concerned with the wider organizational view. On the other hand, traditional use cases contain certain assumptions, including the fact that there is a piece of technology to interact with, and also assumptions about the user interface and the kind of interaction to be designed.

Essential use cases (also referred to sometimes as task cases) represent abstractions from scenarios, i.e. they represent a more general case than a scenario embodies, and try to avoid the assumptions of a traditional use case. An essential use case is a structured narrative consisting of three parts: a name that expresses the overall user intention, a stepped description of user actions, and a stepped description of system responsibilities. This division

between user and system responsibilities can be very helpful during conceptual design when considering task allocation and system scope, i.e. what the user is responsible for and what the system is to do.

An example essential use case based on the visa requirements example given above is shown in [Figure 10.13](#). Note that the steps are more generalized than those in the use case in Section 10.6.2, while they are more structured than the scenario in Section 10.6.1. For example, the second user intention does not say anything about choosing options or system prompts; it simply states that the user supplies the required information. This could be achieved in a variety of ways including scanning a passport, accessing a database of personal information based on fingerprint recognition, and so on. The point is that at the time of creating this essential use case, there is no commitment to a particular interaction design. Essential use cases would normally be developed before the more detailed use case.

retrieveVisa USER INTENTION	SYSTEM RESPONSIBILITY
find visa requirements	
	request destination and nationality
supply required information	
	obtain appropriate visa information
obtain a personal copy of visa information	
	offer information in different formats
choose suitable format	
	provide information in chosen format

Figure 10.13 An essential use case for retrieving visa requirements in the travel organizer

Instead of actors, essential use cases are associated with user roles. An actor could be another system, whereas a user role is a role that a number of different people may play when using the system, so it's not a particular person, and not another system. As with use cases, producing an essential use case begins with identifying user roles.

Activity 10.5

Construct an essential use case 'rentMovie' for the user role 'Subscriber' of the movie rental subscription service discussed in Activity 10.4.

Comment

Show/Hide

10.7 Task Analysis

Task analysis is used mainly to investigate an existing situation, not to envision new products. It is used to analyze the underlying rationale and purpose of what people are doing: what are they trying to achieve, why are they trying to achieve it, and how are they going about it? The information gleaned from task analysis establishes a foundation of existing practices on which to build new requirements or design new tasks.

Task analysis is an umbrella term that covers techniques for investigating cognitive processes and physical actions, at a high level of abstraction and in minute detail. In practice, task analysis techniques have had a mixed reception. The most widely used version is Hierarchical Task Analysis, and this is the technique introduced in this chapter.

10.7.1 Hierarchical Task Analysis

Hierarchical Task Analysis (HTA) was originally designed to identify training needs (Annett and Duncan, 1967). It involves breaking a task down into subtasks and then into sub-subtasks and so on. These are then grouped together as plans that specify how the tasks might be performed in a real situation. HTA focuses on the physical and observable actions that are performed, and includes looking at actions that are not related to software or an interactive product at all. The starting point is a user goal. This is then examined and the main tasks associated with achieving that goal are identified. Where appropriate, these tasks are subdivided into subtasks, and then subtasks can be divided further – down to low-level steps of the interaction which may be represented in a screen sketch.

Consider the task of buying a DVD (based on Hornsby, 2010). This task can be decomposed into the subtasks: locate DVD; add DVD to shopping basket; enter payment details; complete address; and confirm order. Some of these subtasks might not be performed if the user is a regular user – entering payment and address details may not be performed in this case. This can be captured through plans. [Figure 10.14](#) shows the subtasks for buying a DVD and one plan showing two alternative paths through those subtasks.

- 0. In order to buy a DVD
 - 1. locate DVD
 - 2. add DVD to shopping basket
 - 3. enter payment details
 - 4. complete address
 - 5. confirm order

plan 0: If regular user do 1-2-5. If new user do 1-2-3-4-5.

[Figure 10.14](#) An HTA for buying a DVD

An alternative expression of an HTA is a graphical box-and-line notation. [Figure 10.15](#) shows the graphical version of the HTA in

[Figure 10.14](#). Here the subtasks are represented by named boxes with identifying numbers. The hierarchical relationship between tasks is shown using a vertical line. If a task is not decomposed any further then a thick horizontal line is drawn underneath the corresponding box. Plans are also shown in this graphical form. They are written alongside the vertical line emitting from the task being decomposed. Plans are also shown in this graphical form. They are written alongside the vertical line emitting from the task being decomposed.

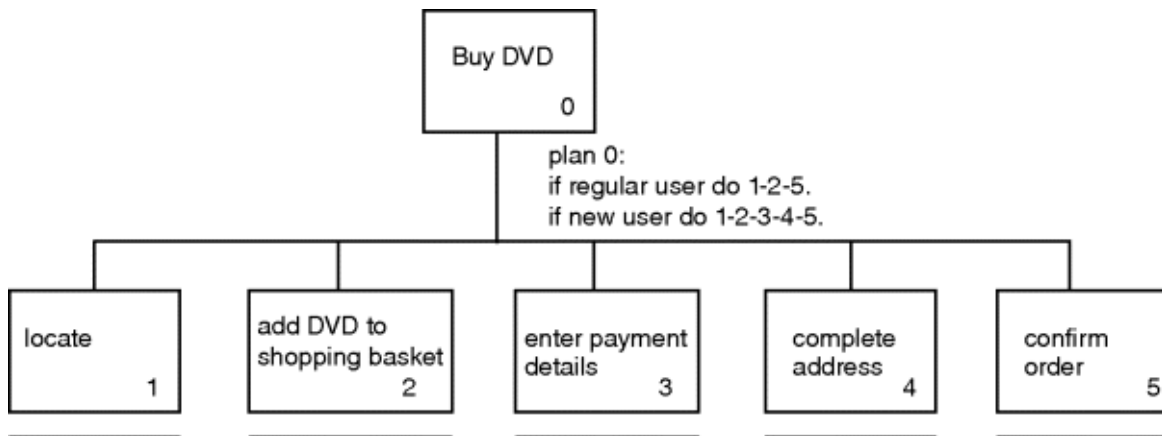


Figure 10.15 A graphical representation of the task analysis for buying a DVD

Use of HTA has been controversial, with both its supporters and its detractors. There are two main problems with using it on real problems:

1. Real tasks are very complex, and task analysis does not scale very well. The notation soon becomes unwieldy, making it difficult to follow.
2. Task analysis is limited in the kinds of task it can model. For example, it cannot model tasks that are overlapping or in parallel, nor can it model interruptions. Most people work through interruptions of various kinds, and many significant tasks happen in parallel.

On the other hand, benefits of task analysis include (Hornsby, 2010):

1. It lets you objectively compare alternative designs, based on a user's planned tasks and subtasks.

2. It provides a good understanding of the interaction at whichever level of abstraction is appropriate. This facilitates good design.
3. It supports design reuse – again at different levels of abstraction.

Activity 10.6

Consider the travel organizer again and perform hierarchical task analysis for the goal of identifying a vacation. Include all plans in your answer. Express the task analysis textually and graphically.

Comment

Show/Hide

Assignment

This assignment is the first of five assignments that together take you through the complete development lifecycle for an interactive product. This assignment requires you to use techniques described in this chapter for establishing requirements. You will also need to draw on techniques from [Chapters 7](#) and [8](#). The further four assignments are at the end of [Chapters 11](#), [12](#), [14](#), and [15](#).

The overall assignment is for you to design and evaluate an interactive product for booking tickets online for events like concerts, the theater, and the cinema. Most venues have an online booking facility already, but it can be awkward and frustrating to identify and book the seats you want.

For this assignment, you should:

- a. Identify users' needs for this online facility. You could do this in a number of ways. For example, you could observe people using ticket agents, think about your own experience of purchasing tickets, look at websites for booking tickets, interview friends and family about their experiences, and so on. Record your data carefully.
- b. Based on your user requirements, choose two different user profiles and produce one persona and one main scenario for each, capturing how the user is expected to interact with the product.
- c. Perform a task analysis on the main task associated with the ticket booking system, i.e. booking a ticket.
- d. Based on this analysis, produce a use case for the main task of booking a ticket.
- e. Using the data gathered in part (a) and your subsequent analysis, identify different kinds of requirements for the

product, according to the headings introduced in Section 10.3.
Write up the requirements in the style of the Volere shell.

Take a Quickvote on Chapter 10:

www.id-book.com/quickvotes/chapter10

Summary

In this chapter, we have looked in more detail at the importance of the requirements activity, and how to establish requirements for interaction design. The data gathering techniques introduced in [Chapter 7](#) can be used in various combinations to gather requirements data. In addition, contextual inquiry, studying documentation, and researching similar products are commonly used techniques. Scenarios, use cases, and essential use cases are helpful techniques for documenting the findings from data gathering sessions. Task analysis is a little more structured, but does not scale well.

Key points

- Getting the requirements right is crucial to the success of the interactive product.
- There are different kinds of requirements: functional, data, environmental (context of use), user characteristics, usability goals, and user experience goals. Every product will have requirements under each of these headings.
- The most commonly used data gathering techniques for this activity are: questionnaires, interviews, focus groups, direct observation, indirect observation, studying documentation, researching similar products, and contextual inquiry.
- Descriptions of user tasks such as scenarios, use cases, and essential use cases help users to articulate existing work practices. They also help to express envisioned use for new products.
- Task analysis techniques help to investigate existing systems and current practices.

Further Reading

DIAPER, D. and STANTON, N. (2004) *The Handbook of Task Analysis for Human–Computer Interaction*. Lawrence Erlbaum Associates. This collection of articles covers the wide diversity of task analysis, including the foundations of task analysis, cognitive approaches, formal notations, and industry experiences. This is not a book to read cover to cover, but is more of a reference book.

GOTTESDIENER, E. (2005) *The Software Requirements Memory Jogger*. Goal/QPC. This handy little book is an excellent practical resource for developing and managing requirements. It is written in a direct and informative style that provides specific guidance including tips, warnings, and factors to consider.

PRUITT, J. and ADLIN, T. (2006) *The Persona Lifecycle: Keeping people in mind throughout product design*. Morgan Kaufmann. This book explains how to use personas in practice – how to integrate them into a product lifecycle, stories from the field, and bright ideas – as well as many example personas. It also includes five guest chapters that place personas in the context of other product design concerns.

ROBERTSON, S. and ROBERTSON, J. (2013) *Mastering the Requirements Process* (3rd edn). Pearson Education. In this book, Robertson and Robertson provide a very practical and useful framework for software requirements work.